

FitGuard API

Complete Developer Reference

For Flutter & AI Integration Teams

Base URL: <https://fitguard-api-production.up.railway.app>

0. How This API Works — Read This First

0.1 Base URL

Every request goes to:

```
https://fitguard-api-production.up.railway.app/api/<endpoint>
```

Example: to login, you call POST <https://fitguard-api-production.up.railway.app/api/auth/login>

0.2 Authentication — JWT Tokens

Most endpoints require you to be logged in. After login you get a JWT token. You must send this token with every protected request inside the HTTP header like this:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

In Flutter: store the token in flutter_secure_storage after login. Add it to every Dio/http request using an interceptor. The token expires in 7 days — user must login again after that.

0.3 User Roles

Every user has exactly one role. The role controls what they can and cannot do:

Role	What they can do
user	Register, manage their own profile, browse coaches, subscribe to one coach, log workout sessions, view their progress, get an AI plan
coach	Everything a user can do, PLUS: assign workout/nutrition plans to their subscribers, view their subscriber list, update their coach bio/specialties. A user becomes a coach only after applying and being approved by an admin.
admin	Everything, PLUS: approve/reject coach applications, manage all users, change user roles, view dashboard metrics. Admins are created manually in the database — there is no public endpoint to become an admin.

IMPORTANT: The database stores the coach role as "trainer" internally, but the API always returns "coach" in JSON responses. Never send "trainer" — always send "coach".

0.4 Every Error Looks The Same

When something goes wrong, every error response has this exact structure, no matter what endpoint:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
```

```

    "message": "age must be an integer between 8 and 110"
  }
}

```

The HTTP status code tells you what category of error it is. The "code" field tells you exactly what went wrong. Here are all possible error codes:

HTTP	code	Meaning
400	VALIDATION_ERROR	You sent wrong/missing data. Check the message field for which field failed.
400	SELF_ROLE_CHANGE	Admin tried to change their own role — not allowed.
400	INVALID_RESET_TOKEN	Password reset token is wrong or has expired (1-hour window).
400	INVALID_TOKEN	Email verification token is invalid.
401	UNAUTHORIZED	No token provided, or token is expired/invalid. Log the user out and redirect to login.
403	FORBIDDEN	You are logged in but your role does not have permission for this action.
403	FORBIDDEN_ROLE	Tried to register with role "coach" or "admin". Not allowed — everyone starts as user.
403	NOT_SUBSCRIBED	Coach tried to assign a plan to a user who is not subscribed to them.
404	NOT_FOUND	The resource (user, coach, exercise, session) does not exist.
409	EMAIL_TAKEN	Tried to register with an email that already exists.
409	ACTIVE_SUBSCRIPTION_EXISTS	User already has an active subscription to a different coach.
409	APPLICATION_PENDING	User already has a pending coach application.

0.5 The User Journey (Big Picture)

Here is how a typical user flows through the whole system from signup to workout tracking:

- User registers → gets JWT token → email verification token sent
- User verifies email via link (contains token)
- User browses coach list → picks a coach → subscribes (1 month)
- Coach logs in → sees their subscriber list → assigns a workout + nutrition plan to the user
- User opens their plan → does exercises → logs each workout session
- User views their progress over time (accuracy, reps, session history)
- Subscription expires → user can re-subscribe to same or different coach

OR the user can skip the coach entirely and use the free AI plan generator instead of step 3-4.

1. Health

Use this to check if the server is alive before making other calls.

GET `/api/health` — *Is the server alive?*

No auth required. No request body. Just call it and check for 200.

Response — 200 OK

```
{ "status": "ok" }
```

2. Auth — Registration & JWT

POST `/api/auth/register` — *Create a new account*

No auth required. Creates a user with a complete fitness profile in one call. Everyone starts with role "user" — you cannot register as coach or admin.

Request Body (application/json)

Field	Details
email *	String. Valid email format. Must be unique in the system.
password *	String. Minimum 8 characters.
role	String. Optional. Always resolves to "user" — any other value throws 403.
profile *	Object. See profile fields table below. All starred fields are required.
profile.name	String. Optional. Display name.
profile.age *	Integer. Between 8 and 110.
profile.heightCm *	Number. Between 80 and 260.
profile.weightKg *	Number. Between 20 and 350.
profile.mealsPerDay *	Integer. Between 1 and 12.
profile.gender *	String. Exactly one of: "Male", "Female"
profile.goal *	String. Exactly one of: "Muscle Building", "Weight Loss", "Maintain"
profile.activityLevel *	String. Exactly one of: "Sedentary", "Light", "Moderate", "Active", "Very Active"
profile.dietaryPreference *	String. Exactly one of: "No restriction", "Vegetarian", "Vegan", "Pescatarian", "Low-carb"
profile.foodDislikes	String. Optional. Free text. Example: "liver, okra"
profile.healthConditions	String. Optional. Free text. Example: "diabetes, high blood pressure"
profile.allergies	String. Optional. Free text. Example: "nuts, lactose, gluten"

Full Request Example

```
{
  "email": "ahmed@example.com",
  "password": "mypassword123",
  "profile": {
    "name": "Ahmed",
```

```

    "age": 25,
    "heightCm": 178,
    "weightKg": 75,
    "mealsPerDay": 3,
    "gender": "Male",
    "goal": "Muscle Building",
    "activityLevel": "Moderate",
    "dietaryPreference": "No restriction",
    "foodDislikes": "okra",
    "healthConditions": "",
    "allergies": ""
  }
}

```

Response — 201 Created

```

{
  "user": {
    "id": "6745abc123def456",
    "email": "ahmed@example.com",
    "role": "user",
    "profile": { ...all profile fields... }
  },
  "devEmailVerificationToken": "a3f9..." // only shown in development mode
}

```

The devEmailVerificationToken is only returned when the server is NOT in production. In production, the token would be emailed. For dev/testing use this token with the verify-email endpoint.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	Any profile field is missing or out of range. Message tells you which field.
403	FORBIDDEN_ROLE	You passed role: "coach" or role: "admin" in the body.
409	EMAIL_TAKEN	An account with this email already exists.

POST /api/auth/login — Login and get JWT token

No auth required. Returns a JWT token — store this and send it with every subsequent request.

Request Body

Field	Details
email *	String. The email used at registration.
password *	String. The password used at registration.

Response — 200 OK

```

{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "6745abc123def456",
    "email": "ahmed@example.com",
    "role": "user",
    "profile": { ...all profile fields... }
  }
}

```

Store the token securely. In Flutter: use `flutter_secure_storage`. The token expires in 7 days. There is no refresh endpoint — when it expires the user must log in again.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	Email or password field is missing.
401	INVALID_CREDENTIALS	Email not found or password is wrong. Both cases return the same error (security best practice — do not reveal which).

POST `/api/auth/forgot-password` — Request a password reset token

No auth required. The user provides their email. The server generates a one-time token valid for 1 hour.

Request Body

Field	Details
email *	String. The account email address.

Response — 200 OK (always 200, even if email not found)

```
{
  "message": "If that email exists, a reset link has been sent.",
  "devToken": "c7f2a8..." // only in development mode
}
```

Security design: The response is always 200 even if the email does not exist. This prevents attackers from discovering which emails are registered. In development, `devToken` is returned directly. In production you would email the token to the user.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	Email field is missing from the request body.

POST `/api/auth/reset-password` — Set a new password using the reset token

No auth required. Use the token from the forgot-password response (or email link in production) to set a new password. Token expires after 1 hour and is deleted after use.

Request Body

Field	Details
token *	String. The token from the forgot-password <code>devToken</code> field (dev) or email link (prod).
newPassword *	String. The new password. Minimum 8 characters.

Response — 200 OK

```
{ "message": "Password reset successful." }
```

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	token or newPassword missing, or newPassword is under 8 characters.
400	INVALID_RESET_TOKEN	Token does not exist or has expired (older than 1 hour).

GET `/api/auth/verify-email?token=<token>` — *Verify the user's email address*

No auth required. The token comes from the `devEmailVerificationToken` field in the register response (dev) or from an email link (production). Once verified, `emailVerifiedAt` is set on the user.

Query Parameter

Field	Details
<code>token *</code>	String. The email verification token from registration.

Response — 200 OK

```
{ "message": "Email verified successfully." }
```

If the email was already verified before:

```
{ "message": "Email already verified." }
```

All Possible Errors

Status	code	When it happens
400	<code>VALIDATION_ERROR</code>	token query parameter is missing.
400	<code>INVALID_TOKEN</code>	Token does not match any user. May have already been used or never existed.

3. Users — Profile Management

GET `/api/users/me/profile` — *Get the logged-in user's profile*

Requires auth token. Returns the current user's role and full profile data.

Response — 200 OK

```
{
  "role": "user",
  "profile": {
    "name": "Ahmed",
    "age": 25,
    "heightCm": 178,
    "weightKg": 75,
    "mealsPerDay": 3,
    "gender": "Male",
    "goal": "Muscle Building",
    "activityLevel": "Moderate",
    "dietaryPreference": "No restriction",
    "foodDislikes": "okra",
    "healthConditions": "",
    "allergies": ""
  }
}
```

All Possible Errors

Status	code	When it happens
401	<code>UNAUTHORIZED</code>	No token or expired token.
404	<code>NOT_FOUND</code>	The user account no longer exists in the database.

PATCH `/api/users/me/profile` — Update the logged-in user's profile

Requires auth token. You only need to send the fields you want to change — all fields are optional. Fields not sent are left unchanged.

Request Body (send only the fields you want to update)

Field	Details
name	String. Display name.
age	Integer. 8–110.
heightCm	Number. 80–260.
weightKg	Number. 20–350.
mealsPerDay	Integer. 1–12.
gender	String. "Male" or "Female".
goal	String. "Muscle Building", "Weight Loss", or "Maintain".
activityLevel	String. "Sedentary", "Light", "Moderate", "Active", or "Very Active".
dietaryPreference	String. "No restriction", "Vegetarian", "Vegan", "Pescatarian", or "Low-carb".
foodDislikes	String. Free text.
healthConditions	String. Free text.
allergies	String. Free text.

Response — 200 OK

Returns the same shape as GET `/api/users/me/profile` with the updated values.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	A field you sent has an invalid value (e.g. age: 200, gender: "Unknown").
401	UNAUTHORIZED	No token or expired token.

POST `/api/users/me/ai-plan` — Generate a free AI workout + nutrition plan

Requires auth token. Generates a basic plan based on the user's profile (goal, weight, height). Deactivates any previous AI plan. This is a stub — real AI integration is pending. Returns a PlanAssignment with source "ai".

Profile must have heightCm, weightKg and goal set before calling this. If any are missing you get a 400 error.

Request Body

None. The server reads the user's profile automatically.

Response — 201 Created

```
{
  "plan": {
    "_id": "6745def789abc",
    "userId": "6745abc123def456",
    "assignedBy": "6745abc123def456",
    "source": "ai",
    "workoutPlan": [
      "3 full-body sessions/week focused on Muscle Building",
```

```

    "Start each session with dynamic warm-up"
  ],
  "nutritionPlan": [
    "Prioritize whole foods and hydration",
    "Set calories for Muscle Building with weekly check-ins"
  ],
  "notes": "Generated from profile inputs. External AI integration pending.",
  "active": true
}
}

```

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	Profile is missing heightCm, weightKg, or goal.
401	UNAUTHORIZED	No token or expired token.

4. Coaches — Applications & Profiles

GET `/api/coaches/public` — *List all approved coaches*

No auth required. Returns all users with the coach role along with their bio and specialties from their coach application.

Response — 200 OK

```

{
  "coaches": [
    {
      "id": "6745abc123def456",
      "email": "coach@example.com",
      "name": "Sara",
      "bio": "Certified personal trainer with 5 years experience.",
      "specialties": ["Weight Loss", "HIIT"],
      "memberSince": "2024-01-15T10:30:00.000Z"
    }
  ]
}

```

GET `/api/coaches/:id` — *Get a single coach's public profile*

No auth required. :id is the coach's user ID (the id field from the coaches list).

Response — 200 OK

```

{
  "coach": {
    "id": "6745abc123def456",
    "email": "coach@example.com",
    "name": "Sara",
    "bio": "Certified personal trainer with 5 years experience.",
    "specialties": ["Weight Loss", "HIIT"],
    "memberSince": "2024-01-15T10:30:00.000Z"
  }
}

```

All Possible Errors

Status	code	When it happens
404	NOT_FOUND	No coach with this ID exists.

POST /api/coaches/applications — Apply to become a coach

Requires auth token. Requires role: user. A user submits a coach application. The admin reviews it. If approved, the user's role changes to "coach". Only one application can be pending at a time.

Request Body

Field	Details
bio	String. Optional. Max 500 characters. Your coaching background and experience.
specialties	Array of strings. Optional. Your specializations. Example: ["HIIT", "Nutrition", "Weight Loss"]

Response — 201 Created

```
{
  "application": {
    "_id": "app123",
    "userId": "6745abc123def456",
    "bio": "Certified trainer...",
    "specialties": ["HIIT", "Nutrition"],
    "status": "pending",
    "decisionNote": "",
    "reviewedBy": null,
    "reviewedAt": null
  }
}
```

All Possible Errors

Status	code	When it happens
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Role is not "user". Coaches cannot re-apply while approved.
409	APPLICATION_PENDING	You already have a pending application. Wait for admin decision.

GET /api/coaches/applications — List all pending coach applications (admin only)

Requires auth token. Requires role: admin. Returns all applications with status "pending".

Response — 200 OK

```
{
  "applications": [
    {
      "_id": "app123",
      "userId": { "_id": "...", "email": "user@example.com", "profile": {...} },
      "bio": "...",
      "specialties": [...],
      "status": "pending"
    }
  ]
}
```

userId is populated — it contains the full user object (email + profile), not just the ID string.

PATCH `/api/coaches/applications/:id/decision` — Approve or reject a coach application (admin only)

Requires auth token. Requires role: admin. :id is the application _id. If approved, the user's role is automatically changed to "coach". If rejected, role stays "user".

Request Body

Field	Details
decision *	String. Exactly "approved" or "rejected". Nothing else.
note	String. Optional. Admin's reason for the decision.

Response — 200 OK

Returns the updated application object with status, reviewedBy, and reviewedAt set.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	decision is not "approved" or "rejected".
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not an admin.
404	NOT_FOUND	Application ID does not exist.

PATCH `/api/coaches/me/profile` — Update coach bio and specialties (coach only)

Requires auth token. Requires role: coach. Updates the coach's own bio and specialties. Only coaches with an approved application can do this.

Request Body

Field	Details
bio	String. Optional. Max 500 characters.
specialties	Array of strings. Optional.

Response — 200 OK

Returns the updated coach application object.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	bio is not a string, or specialties is not an array of strings.
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not a coach.
404	NOT_FOUND	No approved coach application found for this user.

GET `/api/coaches/me/subscribers` — List active subscribers (coach only)

Requires auth token. Requires role: coach. Returns all users currently subscribed to this coach with an active, non-expired subscription.

Response — 200 OK

```
{
```

```

"subscribers": [
  {
    "subscriptionId": "sub123",
    "userId": "user456",
    "email": "user@example.com",
    "name": "Ahmed",
    "startDate": "2025-05-01T00:00:00.000Z",
    "endDate": "2025-06-01T00:00:00.000Z"
  }
]
}

```

All Possible Errors

Status	code	When it happens
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not a coach.

5. Subscriptions — Coach-User Monthly Subscription

A user can subscribe to exactly one coach at a time. Each subscription lasts 1 month. After it expires or is cancelled, the user can subscribe again.

GET `/api/subscriptions/me` — *Get my current active subscription*

Requires auth token. Returns the current active, non-expired subscription, or null if none exists.

Response — 200 OK (has subscription)

```

{
  "subscription": {
    "_id": "sub123",
    "userId": "user456",
    "coachId": "coach789",
    "startDate": "2025-05-01T00:00:00.000Z",
    "endDate": "2025-06-01T00:00:00.000Z",
    "status": "active"
  }
}

```

Response — 200 OK (no active subscription)

```

{ "subscription": null }

```

POST `/api/subscriptions` — *Subscribe to a coach for one month*

Requires auth token. Creates a 1-month subscription starting today. If already subscribed to the SAME coach, returns the existing subscription (no duplicate created). If subscribed to a DIFFERENT coach, returns 409.

Request Body

Field	Details
coachId *	String. The _id or id of the coach from the coaches list.

Response — 201 Created (new subscription)

Returns the full subscription object with startDate and endDate set.

Response — 200 OK (already subscribed to same coach)

Returns the existing active subscription. Nothing changes.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	coachId is missing.
401	UNAUTHORIZED	Not logged in.
404	NOT_FOUND	No coach with this ID exists or is approved.
409	ACTIVE_SUBSCRIPTION_EXISTS	Already subscribed to a different coach. Cancel first.

DELETE /api/subscriptions/me — Cancel my current subscription

Requires auth token. Sets the subscription status to "cancelled" and records the cancellation time. After this, the user can subscribe to any coach again.

Response — 200 OK

Returns the updated subscription object with status: "cancelled" and cancelledAt set.

All Possible Errors

Status	code	When it happens
401	UNAUTHORIZED	Not logged in.
404	NOT_FOUND	No active subscription to cancel.

6. Exercises — Exercise Catalog

Exercises have two types. "tracked" means the CV (computer vision) system counts reps and detects form mistakes. "guided" means the exercise is shown with instructions but not tracked by CV.

GET /api/exercises — List all active exercises

No auth required. Returns all exercises with isActive: true, sorted alphabetically by name. Filter by type using the query parameter.

Query Parameters (optional)

Field	Details
type	String. "tracked" or "guided". Leave empty to get all exercises.

Example calls

```
GET /api/exercises → all exercises
GET /api/exercises?type=tracked → only CV-tracked exercises
GET /api/exercises?type=guided → only guided exercises
```

Response — 200 OK

```
{
  "exercises": [
    {
      "_id": "ex123",
```

```

    "name": "Push-up",
    "type": "tracked",
    "instructions": "Start in plank position...",
    "isActive": true
  }
]
}

```

GET `/api/exercises/:id` — *Get a single exercise by ID*

No auth required.

Response — 200 OK

Returns { "exercise": { ...same fields as above... } }

All Possible Errors

Status	code	When it happens
404	NOT_FOUND	No exercise with this ID.

POST `/api/exercises` — *Create a new exercise (admin only)*

Requires auth token. Requires role: admin.

Request Body

Field	Details
name *	String. Exercise name.
type *	String. "tracked" or "guided".
instructions *	String. How to perform the exercise.

Response — 201 Created

Returns { "exercise": { _id, name, type, instructions, isActive: true } }

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	name, type, or instructions is missing.
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not an admin.

PATCH `/api/exercises/:id` — *Update an exercise (admin only)*

Requires auth token. Requires role: admin. Send only the fields you want to change.

Request Body (all optional)

Field	Details
name	String.
type	String. "tracked" or "guided".
instructions	String.
isActive	Boolean. Set to false to hide the exercise from the list without deleting it.

Response — 200 OK

Returns the updated exercise object.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	type is not "tracked" or "guided".
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not an admin.
404	NOT_FOUND	Exercise ID does not exist.

7. Workouts — Plans & Assignments

A plan (PlanAssignment) is either generated by AI or assigned by a coach. A user has one active plan at a time. When a new plan is assigned or generated, the old one is automatically deactivated.

GET `/api/workouts/me/plan` — *Get my current active plan*

Requires auth token. Returns the user's currently active plan, or null if no plan exists yet.

Response — 200 OK (has plan)

```
{
  "plan": {
    "_id": "plan123",
    "userId": "user456",
    "assignedBy": "coach789",
    "source": "coach", // "ai" or "coach"
    "workoutPlan": [
      "3x10 squats",
      "3x12 push-ups",
      "30 min cardio"
    ],
    "nutritionPlan": [
      "2500 kcal/day",
      "150g protein",
      "Avoid processed sugar"
    ],
    "notes": "Focus on form over weight this week.",
    "active": true,
    "createdAt": "2025-05-01T10:00:00.000Z"
  }
}
```

Response — 200 OK (no plan yet)

```
{ "plan": null }
```

All Possible Errors

Status	code	When it happens
401	UNAUTHORIZED	Not logged in.

POST `/api/workouts/coach/assignments` — *Assign a plan to a subscribed user (coach only)*

Requires auth token. Requires role: coach. The coach selects one of their active subscribers and assigns a workout + nutrition plan. The user must have an active, non-expired subscription to this coach. Any previous coach-assigned plan for that user is deactivated.

Request Body

Field	Details
userId *	String. The ID of the user to assign the plan to. Must be actively subscribed to this coach.
workoutPlan *	Array of strings. Required. Must have at least 1 item. Each string is one instruction/exercise.
nutritionPlan	Array of strings. Optional. Each string is one nutrition guideline.
notes	String. Optional. Free text notes from the coach.

Full Request Example

```
{
  "userId": "6745abc123def456",
  "workoutPlan": [
    "Day 1: 3x10 squats, 3x12 push-ups",
    "Day 2: Rest",
    "Day 3: 30 min cardio, 3x15 lunges"
  ],
  "nutritionPlan": [
    "2500 kcal/day",
    "150g protein",
    "Drink 3L water daily"
  ],
  "notes": "Focus on form. Increase weight next week if squats feel easy."
}
```

Response — 201 Created

Returns the newly created plan assignment object.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	userId missing, workoutPlan is empty/not an array, or nutritionPlan is not an array.
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not a coach.
403	NOT_SUBSCRIBED	The userId you specified is not subscribed to you or subscription has expired.

8. Progress — Workout Sessions & Stats

After a user completes an exercise (using CV tracking), the app logs a WorkoutSession. The session records how many reps were done, how many were correct, and what mistakes were made. This data is used to show the user their progress over time.

POST `/api/progress/sessions` — *Log a completed workout session*

Requires auth token. The CV system calls this after the user finishes an exercise. correctReps + wrongReps MUST equal totalReps exactly, or you get a validation error.

Request Body

Field	Details
exerciseId *	String. The _id of the exercise from the exercises catalog.
totalReps *	Number. Total reps attempted. Must equal correctReps + wrongReps.
correctReps *	Number. Reps performed with correct form.
wrongReps *	Number. Reps performed with incorrect form.
mistakes	Array of objects. Optional. Each object: { "type": "knee cave", "count": 3 }. Lists each distinct mistake and how many times it occurred.

Full Request Example

```
{
  "exerciseId": "ex123",
  "totalReps": 10,
  "correctReps": 8,
  "wrongReps": 2,
  "mistakes": [
    { "type": "knee cave", "count": 1 },
    { "type": "shallow depth", "count": 1 }
  ]
}
```

Response — 201 Created

```
{
  "session": {
    "_id": "sess123",
    "userId": "user456",
    "exerciseId": "ex123",
    "tracked": true,
    "totalReps": 10,
    "correctReps": 8,
    "wrongReps": 2,
    "mistakes": [{ "type": "knee cave", "count": 1 }, ...],
    "sessionAt": "2025-05-08T14:30:00.000Z"
  }
}
```

The "tracked" field is set automatically by the server based on the exercise type — you do not need to send it.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	exerciseId missing, or correctReps + wrongReps ≠ totalReps.
401	UNAUTHORIZED	Not logged in.
404	NOT_FOUND	Exercise with that ID does not exist.

GET /api/progress/sessions/:id — Get a single session by ID

Requires auth token. Returns full session detail with the exercise data populated. You can only view your own sessions — trying to view someone else's session returns 403.

Response — 200 OK

```
{
  "session": {
    "_id": "sess123",
    "userId": "user456",
    "exerciseId": {
      "_id": "ex123",
      "name": "Push-up",
      "type": "tracked",
      "instructions": "Start in plank position..."
    },
    "tracked": true,
    "totalReps": 10,
    "correctReps": 8,
    "wrongReps": 2,
    "mistakes": [...],
    "sessionAt": "2025-05-08T14:30:00.000Z"
  }
}
```

exerciseld here is a populated object (not just a string ID) — it contains the full exercise details.

All Possible Errors

Status	code	When it happens
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	This session belongs to a different user.
404	NOT_FOUND	Session ID does not exist.

GET /api/progress/me — Get progress summary and session history

Requires auth token. Returns a summary (totals + accuracy score) and a paginated list of past sessions. Supports date range filtering.

Query Parameters (all optional)

Field	Details
page	Integer. Page number. Default: 1.
limit	Integer. Sessions per page. Default: 20. Max: 100.
from	String. ISO date string. Only return sessions on or after this date. Example: 2025-01-01
to	String. ISO date string. Only return sessions on or before this date. Example: 2025-12-31

Example calls

```
GET /api/progress/me → last 20 sessions
GET /api/progress/me?page=2&limit=10 → sessions 11-20
GET /api/progress/me?from=2025-05-01&to=2025-05-31 → May 2025 sessions
```

Response — 200 OK

```
{
  "summary": {
```

```

    "totalReps": 250,
    "correctReps": 210,
    "wrongReps": 40,
    "accuracy": 0.84    // 84% = correctReps / totalReps
  },
  "sessions": [
    {
      "_id": "sess123",
      "exerciseId": { "_id": "ex123", "name": "Push-up", "type": "tracked" },
      "totalReps": 10,
      "correctReps": 8,
      "wrongReps": 2,
      "sessionAt": "2025-05-08T14:30:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 47,
    "pages": 3
  }
}

```

The summary is calculated across the filtered date range, not the entire history. So if you filter by from/to, the accuracy shown is only for that period.

All Possible Errors

Status	code	When it happens
401	UNAUTHORIZED	Not logged in.

9. Admin — User Management & Metrics

All admin endpoints require auth token AND role: admin. Any logged-in non-admin user gets 403 FORBIDDEN.

GET `/api/admin/dashboard` — System-wide metrics overview

Response — 200 OK

```

{
  "metrics": {
    "users": 142,
    "coaches": 8,
    "activeSubscriptions": 23,
    "pendingCoachApplications": 3,
    "activeExercises": 15,
    "loggedSessions": 891
  }
}

```

All Possible Errors

Status	code	When it happens
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not an admin.

GET `/api/admin/users` — List all users with pagination and filtering

Query Parameters (all optional)

Field	Details
page	Integer. Default: 1.
limit	Integer. Default: 20. Max: 100.
role	String. Filter by role. "user", "coach", or "admin".
search	String. Search by email (case-insensitive partial match).

Response — 200 OK

```
{
  "users": [
    {
      "id": "user123",
      "email": "user@example.com",
      "role": "user",
      "profile": { ...all profile fields... },
      "createdAt": "2025-01-01T00:00:00.000Z"
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 142, "pages": 8 }
}
```

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	role filter is not one of the valid values.
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not an admin.

GET `/api/admin/users/:id` — Get a single user by ID

Response — 200 OK

Returns { "user": { id, email, role, profile, createdAt } }

All Possible Errors

Status	code	When it happens
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not an admin.
404	NOT_FOUND	User ID does not exist.

PATCH `/api/admin/users/:id/role` — Change a user's role

Requires auth token. Requires role: admin. An admin can promote users to coach or admin, or demote coaches back to user. An admin cannot change their own role.

Request Body

Field	Details
role *	String. "user", "coach", or "admin". Do NOT send "trainer".

Response — 200 OK

Returns the updated user object with the new role.

All Possible Errors

Status	code	When it happens
400	VALIDATION_ERROR	role is "trainer" (use "coach") or not one of the valid values.
400	SELF_ROLE_CHANGE	Admin tried to change their own role. Not allowed.
401	UNAUTHORIZED	Not logged in.
403	FORBIDDEN	Not an admin.
404	NOT_FOUND	User ID does not exist.

10. Data Pipelines — How Data Flows

This section explains the full lifecycle of the most important flows from start to finish.

Pipeline A — User Onboarding

- 1. POST /api/auth/register → user created, devEmailVerificationToken returned
- 2. GET /api/auth/verify-email?token=<token> → emailVerifiedAt set on user
- 3. POST /api/auth/login → JWT token returned, store it
- 4. GET /api/users/me/profile → load profile into app

Email verification is optional — unverified users can still use the app. No endpoints block unverified users currently.

Pipeline B — Coach Subscription & Plan Flow

- 1. GET /api/coaches/public → show coach listing to user
- 2. GET /api/coaches/:id → user views a specific coach profile
- 3. POST /api/subscriptions { coachId } → user subscribes, 1-month subscription created
- 4. Coach: GET /api/coaches/me/subscribers → coach sees the new subscriber
- 5. Coach: POST /api/workouts/coach/assignments { userId, workoutPlan, nutritionPlan } → plan assigned
- 6. User: GET /api/workouts/me/plan → user sees their new plan

Pipeline C — Workout Session (CV Flow)

- 1. GET /api/exercises?type=tracked → get list of tracked exercises
- 2. User performs exercise, CV system counts reps and detects mistakes
- 3. POST /api/progress/sessions { exerciseId, totalReps, correctReps, wrongReps, mistakes } → session saved
- 4. GET /api/progress/me → updated accuracy and history shown to user
- 5. GET /api/progress/sessions/:id → detailed view of a single session

Pipeline D — Password Reset

- 1. POST /api/auth/forgot-password { email } → devToken returned (dev) / email sent (prod)
- 2. POST /api/auth/reset-password { token, newPassword } → password updated, token invalidated
- 3. POST /api/auth/login { email, newPassword } → login with new password

Pipeline E — Becoming a Coach

- 1. POST /api/coaches/applications { bio, specialties } → application submitted with status "pending"
- 2. Admin: GET /api/coaches/applications → admin sees pending applications
- 3. Admin: PATCH /api/coaches/applications/:id/decision { decision: "approved" } → user role changed to "coach"
- 4. User logs in again → JWT now has role "coach" embedded
- 5. Coach: PATCH /api/coaches/me/profile → update bio and specialties anytime

11. Flutter Integration Notes

11.1 Storing the Token

Use `flutter_secure_storage` to store the JWT after login. Never store it in `SharedPreferences` (not encrypted).

```
final storage = FlutterSecureStorage();
await storage.write(key: "jwt", value: token);
final token = await storage.read(key: "jwt");
```

11.2 Attaching the Token to Every Request

Add an interceptor to Dio so every request automatically includes the Authorization header:

```
dio.interceptors.add(InterceptorsWrapper(
  onRequest: (options, handler) async {
    final token = await storage.read(key: "jwt");
    if (token != null) {
      options.headers["Authorization"] = "Bearer $token";
    }
    return handler.next(options);
  },
));
```

11.3 Handling 401 — Token Expired

When you get a 401 UNAUTHORIZED response, the token has expired (7-day lifetime). Clear the stored token and redirect the user to the login screen.

```
if (response.statusCode == 401) {
  await storage.delete(key: "jwt");
  Navigator.pushReplacementNamed(context, "/login");
}
```

11.4 Parsing Errors

All error responses have the same structure. Parse the `error.code` field to show the right UI message:

```
try {
  final response = await dio.post("/api/auth/login", data: body);
} on DioException catch (e) {
  final code = e.response?.data["error"]["code"];
  final message = e.response?.data["error"]["message"];
  if (code == "INVALID_CREDENTIALS") {
    // show wrong password UI
  } else if (code == "VALIDATION_ERROR") {
    // show field error UI with message
  }
}
```

```
}
```

11.5 Date Fields

All date fields are ISO 8601 strings in UTC. Use `DateTime.parse()` to convert them:

```
final endDate = DateTime.parse(subscription["endDate"]);
final daysLeft = endDate.difference(DateTime.now()).inDays;
```

11.6 Checking if User is a Coach

After login or profile fetch, check the role field:

```
final role = userData["role"]; // "user", "coach", or "admin"
if (role == "coach") { showCoachDashboard(); }
```

The API always returns "coach", never "trainer". You will never see "trainer" in any response.

12. AI / CV Integration Notes

12.1 Which Exercises Use CV

Only exercises with type: "tracked" are tracked by the computer vision system. Exercises with type: "guided" just show instructions — no rep counting or form detection.

```
// When building the workout UI, check:
if (exercise["type"] == "tracked") {
  // activate CV camera and rep counter
} else {
  // show instructions only
}
```

12.2 What to Log After a CV Session

After the CV system finishes tracking, collect the data and POST to `/api/progress/sessions`. The mistakes array is the most important field — it tells the user exactly what they did wrong.

The rule: `correctReps + wrongReps` MUST equal `totalReps`. The server validates this and rejects the session if it does not add up.

12.3 The Mistakes Structure

Each mistake entry has a type (what went wrong) and a count (how many times). The type is a free-form string — your CV system defines what these are. Examples:

```
"mistakes": [
  { "type": "knee cave", "count": 2 },
  { "type": "shallow squat", "count": 1 },
  { "type": "forward lean", "count": 3 }
]
```

12.4 The AI Plan Endpoint

POST `/api/users/me/ai-plan` generates a basic plan from the user's profile. It is currently a stub — the plans are hardcoded templates, not real AI output. The external AI integration is planned for a future version. The endpoint structure and response shape are final, so you can integrate against it now.

12.5 Using Progress Data to Improve AI

GET /api/progress/me returns accuracy and session history for the logged-in user. For AI training or personalisation, the key fields are:

- summary.accuracy — overall form score (0.0 to 1.0)
- sessions[].mistakes — per-session breakdown of form errors
- sessions[].exerciseld.name — which exercise was done
- sessions[].sessionAt — timestamp for time-series analysis

Use the from and to query parameters to fetch a specific training period for analysis.

13. Quick Reference — All Endpoints

Method	Endpoint	Auth	Description
GET	/api/health	None	Server liveness check
POST	/api/auth/register	None	Create account with fitness profile
POST	/api/auth/login	None	Login → get JWT token
POST	/api/auth/forgot-password	None	Request password reset token
POST	/api/auth/reset-password	None	Set new password using token
GET	/api/auth/verify-email?token=	None	Verify email address
GET	/api/users/me/profile	user+	Get current user profile
PATCH	/api/users/me/profile	user+	Update current user profile
POST	/api/users/me/ai-plan	user+	Generate free AI workout plan
GET	/api/coaches/public	None	List all approved coaches
GET	/api/coaches/:id	None	Get single coach profile
POST	/api/coaches/applications	user	Apply to become a coach
GET	/api/coaches/applications	admin	List pending applications
PATCH	/api/coaches/applications/:id/decision	admin	Approve or reject application
PATCH	/api/coaches/me/profile	coach	Update bio and specialties
GET	/api/coaches/me/subscribers	coach	List active subscribers

GET	/api/subscriptions/me	user+	Get my active subscription
POST	/api/subscriptions	user+	Subscribe to a coach
DELETE	/api/subscriptions/me	user+	Cancel subscription
GET	/api/exercises	None	List exercises (filter by type)
GET	/api/exercises/:id	None	Get single exercise
POST	/api/exercises	admin	Create new exercise
PATCH	/api/exercises/:id	admin	Update exercise
GET	/api/workouts/me/plan	user+	Get my current active plan
POST	/api/workouts/coach/assignments	coach	Assign plan to subscriber
POST	/api/progress/sessions	user+	Log a workout session
GET	/api/progress/sessions/:id	user+	Get single session detail
GET	/api/progress/me	user+	Get progress summary + history
GET	/api/admin/dashboard	admin	System metrics
GET	/api/admin/users	admin	List all users (paginated)
GET	/api/admin/users/:id	admin	Get user by ID
PATCH	/api/admin/users/:id/role	admin	Change user role

Auth column: "None" = no token needed. "user+" = any logged-in user. "user" = user role only. "coach" = coach role only. "admin" = admin role only.